

Университет ИТМО

Факультет программной инженерии и компьютерной техники
Образовательная программа системное и прикладное
программное обеспечение

Лабораторная работа №2
По дисциплине "Вычислительная математика"
Вариант №2

Выполнил студент группы Р3209
Евграфов Артём Андреевич
Проверил:
Рыбаков Степан Дмитриевич

Санкт-Петербург 2026

Содержание

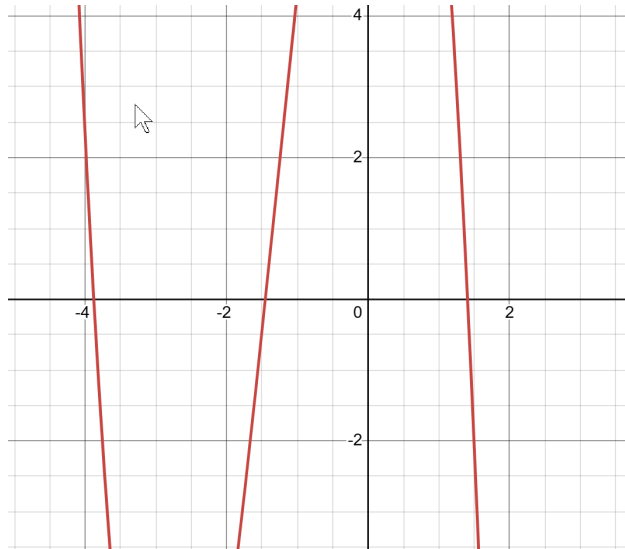
1. Цель работы	2
2. Часть 1	2
3. Листинг программы1	3
4. Часть 2	11
4.1. Графическое отделение корней	11
4.2. Проверка решения	12
4.3. Листинг программы2	13
5. Выводы	16

1. Цель работы

Изучить численные методы решения нелинейных уравнений и реализовать три из них средствами программирования. Понять их сходства и различия.

2. Часть 1

Многочлен $f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95$.



Координаты точки пересечения хорды, проходящей через точки $A(a, f(a))$, $B(b, f(b))$ и прямой Ox : $(a - \frac{b-a}{f(b)-f(a)}f(a), 0)$.

№ шага	a	b	x	$f(a)$	$f(b)$	$f(x)$	$ x_{k+1} - x_k $
1	-4.000	-3.000	-3.785	2.270	-8.280	-1.553	0.785
2	-4.000	-3.785	-3.872	2.270	-1.553	-0.270	0.087
3	-4.000	-3.872	-3.888	2.270	-0.270	-0.044	0.015
4	-4.000	-3.888	-3.890	2.270	-0.044	-0.007	0.003

Таблица 1: Уточнение левого корня методом хорд

$x_{i+1} = x_i - \frac{x_i - x_{i-1}}{f(x_i) - f(x_{i-1})}f(x_i)$. Коэффициент сжатия q : 0.652, усиленное условие: $|x_{k+1} - x_k| < \frac{1-q}{q}\epsilon$.

№ шага	x_{k-1}	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	-2.000	-1.000	-1.473	-0.187	0.473
2	-1.000	-1.473	-1.454	-0.005	0.019
3	-1.473	-1.454	-1.454	0.000	0.000

Таблица 2: Уточнение центрального корня методом секущих

$x_i = \phi(x_{i-1})$, начальное приближение: $x_0 = 1.5$.

№ шага	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	1.500	1.443	-1.488	0.057
2	1.443	1.401	-1.045	0.042
3	1.401	1.372	-0.720	0.029
4	1.372	1.351	-0.490	0.020
5	1.351	1.338	-0.331	0.014
6	1.338	1.328	-0.222	0.009

Таблица 3: Уточнение правого корня методом простой итерации

3. Листинг программы¹

```

1  #include <bits/stdc++.h>
2  #include "matplotlibcpp.h"
3
4  using namespace std;
5  namespace plt = matplotlibcpp;
6
7  void plotFunction(double a, double b, const std::function<double(double)>& f, const
   ↪  std::string& func_name = "") {
8      const int points = 1000;
9      vector<double> x(points), y(points);
10     double step = (b - a) / (points - 1);
11
12     for (int i = 0; i < points; i++) {
13         x[i] = a + i * step;
14         y[i] = f(x[i]);
15     }
16
17     plt::plot(x, y);
18     plt::title(func_name.empty() ? "График функции" : func_name);
19     plt::xlabel("x");
20     plt::ylabel("f(x)");
21     plt::grid(true);
22     plt::show();
23 }
24
25 struct Equation {
26     function<double(double)> f;
27     function<double(double)> df;
28     string view;
29 };
30
31 function<double(double)> current_f;
32 function<double(double)> current_df;
33 double lambda_param = 0;
34
35
36 double f1(double x) {
37     return -1.38 * pow(x, 3) - 5.42 * pow(x, 2) + 2.57 * x + 10.95;
38 }
39 double df1(double x) {
40     return -4.14 * pow(x, 2) - 10.84 * x + 2.57;
41 }
42
43 double f2(double x) {
44     return pow(x, 2) + sin(x) - 1;

```

```

45 }
46 double df2(double x) {
47     return 2 * x + cos(x);
48 }
49
50 double f3(double x) {
51     return exp(x) - 3 * x;
52 }
53 double df3(double x) {
54     return exp(x) - 3;
55 }
56
57 double f(double x) {
58     return current_f(x);
59 }
60 double df(double x) {
61     return current_df(x);
62 }
63 double phi(double x) {
64     return x + lambda_param * f(x);
65 }
66
67 void secantMethod(double a, double b, double epsilon, ostream& out) {
68     double x_prev = a;
69     double x_curr = b;
70
71     int step = 1;
72     double x_next;
73     double diff = b - a;
74
75     out << "--- Метод секущих ---" << "\n";
76     out << setw(5) << "Step"
77         << setw(12) << "x_(k-1)"
78         << setw(12) << "x_k"
79         << setw(12) << "x_(k+1)"
80         << setw(12) << "f(x_(k+1))"
81         << setw(12) << "|diff|" << "\n";
82
83     while (abs(diff) > epsilon) {
84         double f_curr = f(x_curr);
85         double f_prev = f(x_prev);
86
87         if (abs(f_curr - f_prev) < 1e-9) {
88             out << "Деление на 0 => метод сошелся" << "\n";
89             break;
90         }
91
92         x_next = x_curr - (x_curr - x_prev) / (f_curr - f_prev) * f_curr;
93
94         diff = x_next - x_curr;
95         out << setw(5) << step
96             << setw(12) << fixed << setprecision(3) << x_prev
97             << setw(12) << x_curr
98             << setw(12) << x_next
99             << setw(12) << f(x_next)
100             << setw(12) << abs(diff) << "\n";
101
102         x_prev = x_curr;
103         x_curr = x_next;
104         step++;
105

```

```

106         if (step > 1000) {
107             out << "Превышено число итераций\n";
108             break;
109         }
110     }
111
112     out << "Найденный корень: " << x_curr << "\n";
113     out << "Значение функции: " << f(x_curr) << "\n";
114     out << "Число итераций: " << step - 1 << "\n\n";
115 }
116
117 void bisectionMethod(double a, double b, double eps, ostream& out) {
118     out << "--- Метод половинного деления (бисекции) ---\n";
119     out << "Mar |      a      |      b      |      x      |      f(a)      |      f(b)      |      f(x)
120     ↪ | |b-a|  \n";
121     out <<
122     ↪ "-----\n";
123     out << fixed << setprecision(6);
124
125     double fa = f(a);
126     double fb = f(b);
127
128     int step = 0;
129     double x;
130     double fx;
131
132     while ((b - a) > eps) {
133         x = (a + b) / 2.0;
134         fx = f(x);
135
136         out << setw(3) << ++step << " | "
137         << setw(9) << a << " | "
138         << setw(9) << b << " | "
139         << setw(9) << x << " | "
140         << setw(9) << fa << " | "
141         << setw(9) << fb << " | "
142         << setw(9) << fx << " | "
143         << setw(9) << (b - a) << "\n";
144
145         if (abs(fx) < eps) { \
146             break;
147         }
148
149         if (fa * fx < 0) {
150             b = x;
151             fb = fx;
152         } else {
153             a = x;
154             fa = fx;
155         }
156
157         if (step > 1000) {
158             out << "Превышено число итераций.\n";
159             break;
160         }
161     }
162
163     x = (a + b) / 2.0;
164     out <<
165     ↪ "-----\n";
166     out << "Найденный корень: " << x << "\n";

```

```

164     out << "Значение функции: " << f(x) << "\n";
165     out << "Число итераций: " << step << "\n\n";
166 }
167
168 void chordMethod(double a, double b, double eps, ostream& out) {
169     out << "--- Метод хорд ---\n";
170     out << "Step | a      | b      | x      | f(a)   | f(b)   | f(x)   | |x_next -
    ↪ x_prev|\n";
171     out <<
    ↪ "-----\n";
172     out << fixed << setprecision(3);
173
174     int step = 1;
175     double x = (a + b) / 2;
176     double x_prev = a;
177
178     double diff = eps + 1.0;
179
180     while (diff > eps) {
181         if (abs(f(b) - f(a)) < 1e-9) {
182             out << "Деление на 0 => метод сошелся\n";
183             break;
184         }
185
186         x = a - (f(a) * (b - a)) / (f(b) - f(a));
187
188         double fa = f(a);
189         double fb = f(b);
190         double fx = f(x);
191
192         if (step == 1) diff = abs(b - a);
193         else diff = abs(x - x_prev);
194
195         out << step << "      | "
196             << a << " | "
197             << b << " | "
198             << x << " | "
199             << fa << " | "
200             << fb << " | "
201             << fx << " | "
202             << diff << "\n";
203
204         if (fa * fx < 0) {
205             b = x;
206         } else {
207             a = x;
208         }
209
210         x_prev = x;
211         step++;
212
213         if (step > 1000) {
214             out << "Превышено число итераций\n";
215             break;
216         }
217     }
218     out << "Найденный корень: " << x << "\n";
219     out << "Значение функции: " << f(x) << "\n";
220     out << "Число итераций: " << step - 1 << "\n\n";
221 }
222

```

```

223 void newtonMethod(double a, double b, double eps, ostream& out) {
224     out << "--- Метод Ньютона ---\n";
225
226     double fa = f(a);
227     double fb = f(b);
228     double x_k;
229
230     if (abs(fa) < abs(fb))
231         x_k = a;
232     else
233         x_k = b;
234
235     out << "Начальное приближение x0 = " << x_k << " (выбрано как конец с меньшим
    ↳ |f(x)|)\n";
236     out <<
    ↳ "-----\n";
237     out << "№ итер |      x_k      |      f(x_k)      |      f'(x_k)      |      x_{k+1}      |
    ↳ |x_{k+1}-x_k|\n";
238     out <<
    ↳ "-----\n";
239     out << fixed << setprecision(6);
240
241     int step = 0;
242     double x_next;
243     double diff;
244     do {
245         double f_k = f(x_k);
246         double df_k = df(x_k);
247
248         if (abs(df_k) < 1e-12) {
249             out << "Метод не сходится.\n";
250             return;
251         }
252
253         x_next = x_k - f_k / df_k;
254         diff = abs(x_next - x_k);
255         step++;
256
257         out << setw(5) << step << " | "
258             << setw(11) << x_k << " | "
259             << setw(11) << f_k << " | "
260             << setw(11) << df_k << " | "
261             << setw(11) << x_next << " | "
262             << setw(12) << diff << "\n";
263
264         if (diff < eps) break;
265
266         x_k = x_next;
267
268         if (step > 1000) {
269             out << "Превышено число итераций.\n";
270             break;
271         }
272     } while (true);
273
274     out <<
    ↳ "-----\n";
275     out << "Найденный корень: " << x_next << "\n";
276     out << "Значение функции: " << f(x_next) << "\n";
277     out << "Число итераций: " << step << "\n\n";

```



```

279 }
280
281
282 void simpleIterationMethod(double a, double b, double eps, ostream& out) {
283     out << "---- Метод простой итерации ----\n";
284
285     double mid = (a + b) / 2.0;
286     double max_df = max({abs(df(a)), abs(df(b)), abs(df(mid))}); // ?
287     double sign = (df(mid) > 0) ? 1.0 : -1.0;
288     lambda_param = -sign / max_df;
289
290     // q = max |1 + lambda * f'(x)| \approx 0
291     double q_a = abs(1 + lambda_param * df(a));
292     double q_b = abs(1 + lambda_param * df(b));
293     double q_mid = abs(1 + lambda_param * df(mid));
294     double q = max({q_a, q_b, q_mid});
295
296     out << "Интервал: [" << a << "; " << b << "]\n";
297     out << "max|f'(x)| (approx): " << max_df << "\n";
298     out << "Вычисленная lambda: " << lambda_param << "\n";
299     out << "Коэффициент сжатия q: " << q << "\n";
300
301     if (q >= 1.0) {
302         out << "Условие сходимости не выполнено. Метод может разойтись.\n";
303     }
304
305     double stop_coef = 1.0;
306     if (q > 0.5 && q < 1.0) {
307         stop_coef = (1 - q) / q;
308         out << "Так как q > 0.5, используем множитель (1-q)/q = " << stop_coef << "\n";
309     } else if (q <= 0.5) {
310         out << "Так как q <= 0.5, множитель равен 1.\n";
311     }
312
313     out << "-----\n";
314     out << "No | x_k      | x_{k+1}    | f(x_{k+1})  | |x_{k+1} - x_k|\n";
315     out << "-----\n";
316     out << fixed << setprecision(5);
317
318     double x_k = (a + b) / 2.0;
319     double x_next;
320     double diff;
321     int step = 0;
322
323     do {
324         step++;
325         x_next = phi(x_k);
326         diff = abs(x_next - x_k);
327
328         out << step << " | "
329             << x_k << " | "
330             << x_next << " | "
331             << f(x_next) << " | "
332             << diff << "\n";
333
334         if (diff < eps * stop_coef) {
335             break;
336         }
337
338         x_k = x_next;
339

```

```

340     if (step > 1000) { // пусть будет так
341         out << "Превышено число итераций\n";
342         break;
343     }
344
345     // вылетит в бесконечность при расхождении
346     if (abs(x_next) > 1e10) {
347         out << "Метод расходится.\n";
348         return;
349     }
350
351     } while (true);
352
353     out << "-----\n";
354     out << "Найденный корень: " << x_next << "\n";
355     out << "Значение функции: " << f(x_next) << "\n";
356     out << "Число итераций: " << step << "\n\n";
357 }
358
359 bool validateInterval(double a, double b, ostream& out) {
360     if (a >= b) {
361         out << "Некорректные границы интервала (a >= b).\n";
362         return false;
363     }
364
365     double fa = f(a);
366     double fb = f(b);
367
368     if (fa * fb > 0) {
369         out << "На концах интервала функция имеет одинаковые знаки.\n";
370         return false;
371     }
372
373     return true;
374 }
375
376 int main() {
377     vector<Equation> equations = {
378         {f1, df1, "-1.38x^3 - 5.42x^2 + 2.57x + 10.95"},
379         {f2, df2, "x^2 + sin(x) - 1"},
380         {f3, df3, "e^x - 3x"}
381     };
382
383     int inputChoice;
384     cout << "Выберите способ ввода данных:\n1. Консоль\n2. Файл (input.txt)\n> ";
385     while(!(cin >> inputChoice) || inputChoice < 1 || inputChoice > 2) {
386         cin.clear();
387         cin.ignore(numeric_limits<streamsize>::max(), '\n');
388         cout << "Введите 1 или 2\n";
389         cout << "> ";
390     };
391
392     istream* in = &cin;
393     ifstream inFile;
394     if (inputChoice == 2) {
395         inFile.open("input.txt");
396         if (!inFile) {
397             cerr << "Не удалось открыть файл input.txt. Переход на консольный ввод.\n";
398             inputChoice = 1;
399         } else {
400             in = &inFile;

```

```

401     }
402 }
403
404 int outputChoice;
405 cout << "Выберите способ вывода результата:\n1. Консоль\n2. Файл (output.txt)\n> ";
406 while(!(cin >> outputChoice) || outputChoice < 1 || outputChoice > 2) {
407     cin.clear();
408     cin.ignore(numeric_limits<streamsize>::max(), '\n');
409     cout << "Введите 1 или 2\n";
410     cout << "> ";
411 };
412
413 ostream* out = &cout;
414 ofstream outFile;
415 if (outputChoice == 2) {
416     outFile.open("output.txt");
417     if (!outFile) {
418         cerr << "Не удалось открыть файл output.txt. Переход на консольный
↳ вывод.\n";
419         outputChoice = 1;
420     } else {
421         out = &outFile;
422     }
423 }
424
425 if (inputChoice == 1) {
426     cout << "Выберите уравнение:\n";
427     for (int i = 0; i < equations.size(); ++i) {
428         cout << i + 1 << ". " << equations[i].view << "\n";
429     }
430     cout << "> ";
431 }
432
433 int eqIndex;
434
435 while (!(cin >> eqIndex) || eqIndex < 1 || eqIndex > equations.size()) {
436     *out << "Неправильно набран номер, перезвоните позже\n";
437     in->clear();
438     in->ignore(numeric_limits<streamsize>::max(), '\n');
439     *out << "> ";
440 }
441
442 current_f = equations[eqIndex - 1].f;
443 current_df = equations[eqIndex - 1].df;
444
445 if (inputChoice == 1) {
446     cout << "Выберите метод:\n1. Метод хорд\n2. Метод секущих\n3. Метод простой
↳ итерации.\n4. Метод бисекций.\n5. Метод Ньютона\n> ";
447 }
448 int methodIndex;
449
450 while (!(cin >> methodIndex) || methodIndex < 1 || methodIndex > 4) {
451     *out << "Неправильно набран номер, перезвоните позже\n";
452     in->clear();
453     in->ignore(numeric_limits<streamsize>::max(), '\n');
454     *out << "> ";
455 }
456
457 if (inputChoice == 1) cout << "Введите границы интервала a и b: ";
458 double a, b;
459 while (!(cin >> a) || !(cin >> b)) {

```

```

460     *out << "Ну емае\n";
461     in->clear();
462     in->ignore(numeric_limits<streamsize>::max(), '\n');
463     *out << "> ";
464 }
465
466 if (inputChoice == 1) cout << "Введите погрешность вычисления: ";
467 double eps;
468 while (!(*in >> eps)) {
469     *out << "Ну емае\n";
470     in->clear();
471     in->ignore(numeric_limits<streamsize>::max(), '\n');
472     *out << "> ";
473 }
474
475 *out << "Выбрано уравнение: " << equations[eqIndex - 1].view << "\n";
476 *out << "Интервал: [" << a << ", " << b << "], Погрешность: " << eps << "\n\n";
477
478 if (!validateInterval(a, b, *out)) {
479     return 1;
480 }
481
482 switch (methodIndex) {
483     case 1:
484         chordMethod(a, b, eps, *out);
485         break;
486     case 2:
487         secantMethod(a, b, eps, *out);
488         break;
489     case 3:
490         simpleIterationMethod(a, b, eps, *out);
491         break;
492     case 4:
493         bisectionMethod(a, b, eps, *out);
494     case 5:
495         newtonMethod(a, b, eps, *out);
496     default:
497         return 1;
498 }
499
500 plotFunction(a, b, current_f, equations[eqIndex - 1].view);
501
502 if (inputChoice == 2) inFile.close();
503 if (outputChoice == 2) outFile.close();
504
505 return 0;
506 }

```

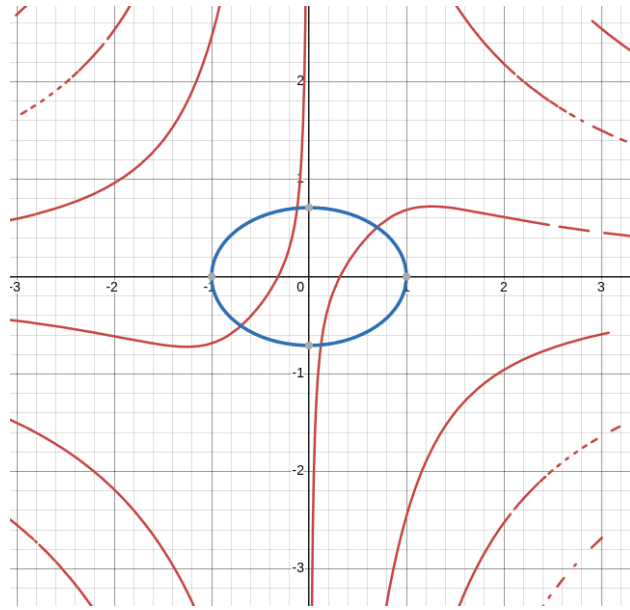
4. Часть 2

Требуется решить систему нелинейных уравнений методом Ньютона с точностью до 0.01:

$$\begin{cases} \tan(xy + 0.1) = x^2, \\ x^2 + 2y^2 = 1. \end{cases}$$

4.1. Графическое отделение корней

Для определения начального приближения построим графики функций, задающих систему. Первое уравнение представим как неявную функцию $\tan(xy + 0.1) - x^2 = 0$, второе — эллипс $x^2 + 2y^2 = 1$. Их



пересечения соответствуют решениям системы.

Выберем корень в первой четверти, где $x \approx 0.7$, $y \approx 0.5$.

Для системы уравнений $\mathbf{F}(x, y) = 0$ с

$$\mathbf{F}(x, y) = \begin{pmatrix} f_1(x, y) \\ f_2(x, y) \end{pmatrix}, \quad f_1 = \tan(xy + 0.1) - x^2, \quad f_2 = x^2 + 2y^2 - 1,$$

итерационная формула метода Ньютона имеет вид

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - J^{-1}(\mathbf{x}^{(k)}) \mathbf{F}(\mathbf{x}^{(k)}),$$

где J — матрица Якоби:

$$J(x, y) = \begin{pmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{pmatrix} = \begin{pmatrix} \frac{y}{\cos^2(xy + 0.1)} - 2x & \frac{x}{\cos^2(xy + 0.1)} \\ 2x & 4y \end{pmatrix}.$$

Начальное приближение: $x_0 = 0.7$, $y_0 = 0.5$ (выбрано по графику).

Процесс уточнения продолжаем до выполнения условия $\max(|x_{k+1} - x_k|, |y_{k+1} - y_k|) < \varepsilon = 0.01$.

Результаты итераций приведены в таблице 4.

№ итерации	x_k	y_k	$f_1(x_k, y_k)$	$f_2(x_k, y_k)$	Δx	Δy
0	0.700	0.500	-0.0070	-0.0100	—	—
1	0.698	0.506	0.0007	0.0001	-0.0019	0.0064

Таблица 4: Уточнение корня методом Ньютона

На первой итерации получены значения $x_1 = 0.698$, $y_1 = 0.506$. Норма разности $\max(|\Delta x|, |\Delta y|) = 0.0064 < 0.01$, поэтому требуемая точность достигнута. Дальнейшие итерации не требуются.

4.2. Проверка решения

Подставим найденные значения в исходную систему:

$$x_1 y_1 + 0.1 = 0.698 \cdot 0.506 + 0.1 = 0.353 + 0.1 = 0.453,$$

$$\tan(0.453) \approx 0.488, \quad x_1^2 = 0.487, \quad \text{разность } 0.001;$$

$$x_1^2 + 2y_1^2 = 0.487 + 2 \cdot 0.256 = 0.487 + 0.512 = 0.999 \approx 1.$$

Невязки составляют 0.001 и -0.001 , что значительно меньше заданной точности. Следовательно, решение найдено верно.

4.3. Листинг программы2

```
1  #include <bits/stdc++.h>
2  #include "matplotlibcpp.h"
3
4  namespace plt = matplotlibcpp;
5  using namespace std;
6
7  struct System {
8      string description;
9      double (*f1)(double, double);
10     double (*f2)(double, double);
11     double (*df1dx)(double, double);
12     double (*df1dy)(double, double);
13     double (*df2dx)(double, double);
14     double (*df2dy)(double, double);
15 };
16
17 double f1_sys1(double x, double y) { return tan(x * y + 0.1) - x * x; }
18
19 double f2_sys1(double x, double y) { return x * x + 2 * y * y - 1; }
20
21 double df1dx_sys1(double x, double y) {
22     double denom = cos(x * y + 0.1);
23     return y / (denom * denom) - 2 * x;
24 }
25
26 double df1dy_sys1(double x, double y) {
27     double denom = cos(x * y + 0.1);
28     return x / (denom * denom);
29 }
30
31 double df2dx_sys1(double x, double y) { return 2 * x; }
32
33 double df2dy_sys1(double x, double y) { return 4 * y; }
34
35 double f1_sys2(double x, double y) { return cos(x - 1) + y - 0.5; }
36
37 double f2_sys2(double x, double y) { return x - cos(y) - 3; }
38
39 double df1dx_sys2(double x, double y) { return -sin(x - 1); }
40
41 double df1dy_sys2(double x, double y) { return 1; }
42
43 double df2dx_sys2(double x, double y) { return 1; }
44
45 double df2dy_sys2(double x, double y) { return sin(y); }
46
47 void plotSystem(const System& sys, double xmin, double xmax, double ymin, double ymax) {
48     const int N = 500;
49     vector<double> xs(N*N), ys(N*N), f1vals(N*N), f2vals(N*N);
50     int idx = 0;
51     double stepx = (xmax - xmin) / (N - 1);
52     double stepy = (ymax - ymin) / (N - 1);
53
54     for (int i = 0; i < N; i++) {
55         double x = xmin + i * stepx;
56         for (int j = 0; j < N; ++j) {
57             double y = ymin + j * stepy;
58             xs[idx] = x;
59             ys[idx] = y;
```

```

60         f1vals[idx] = sys.f1(x, y);
61         f2vals[idx] = sys.f2(x, y);
62         idx++;
63     }
64 }
65
66 plt::figure();
67 vector<double> f1x, f1y;
68 for (int k = 0; k < N*N; ++k) {
69     if (abs(f1vals[k]) < 0.05) {
70         f1x.push_back(xs[k]);
71         f1y.push_back(ys[k]);
72     }
73 }
74 plt::scatter(f1x, f1y, 1.0, {"color", "red"}, {"label", "f1 = 0"});
75
76 vector<double> f2x, f2y;
77 for (int k = 0; k < N*N; ++k) {
78     if (abs(f2vals[k]) < 0.05) {
79         f2x.push_back(xs[k]);
80         f2y.push_back(ys[k]);
81     }
82 }
83 plt::scatter(f2x, f2y, 1.0, {"color", "blue"}, {"label", "f2 = 0"});
84
85 plt::xlabel("x");
86 plt::ylabel("y");
87 plt::title("Графики функций системы");
88 plt::legend();
89 plt::grid(true);
90 plt::show();
91 }
92
93 void newtonSystem(const System& sys, double x0, double y0, double eps, ostream& out) {
94     out << "--- Метод Ньютона для системы ---\n";
95
96     out << "Итерация |      x      |      y      |      f1      |      f2      |      dx
97     ↪ |      dy      \n";
98     out <<
99     ↪ "-----\n";
100     out << fixed << setprecision(6);
101
102     double x = x0, y = y0;
103     int iter = 0;
104     double dx, dy;
105
106     do {
107         double f1v = sys.f1(x, y);
108         double f2v = sys.f2(x, y);
109
110         double J11 = sys.df1dx(x, y);
111         double J12 = sys.df1dy(x, y);
112         double J21 = sys.df2dx(x, y);
113         double J22 = sys.df2dy(x, y);
114
115         double det = J11 * J22 - J12 * J21;
116         if (abs(det) < 1e-12) {
117             out << "Определитель Якоби близок к нулю.\n";
118             return;
119         }
120         dx = -(f1v * J22 - J12 * f2v) / det;

```

```

119     dy = -(J11 * f2v - f1v * J21) / det;
120
121     x += dx;
122     y += dy;
123     iter++;
124
125     out << setw(5) << iter << "      | "
126         << setw(10) << x << " | "
127         << setw(10) << y << " | "
128         << setw(10) << sys.f1(x, y) << " | "
129         << setw(10) << sys.f2(x, y) << " | "
130         << setw(10) << dx << " | "
131         << setw(10) << dy << "\n";
132
133     if (max(abs(dx), abs(dy)) < eps) break;
134     if (iter > 1000) {
135         out << "Достигнут лимит итераций.\n";
136         break;
137     }
138 } while (true);
139
140 out <<
141 → "-----\n";
142 out << "Решение: x = " << x << ", y = " << y << "\n";
143 out << "Значения функций: f1 = " << sys.f1(x, y) << ", f2 = " << sys.f2(x, y) <<
144 → "\n";
145 out << "Количество итераций: " << iter << "\n";
146 out << "Вектор погрешностей последней итерации: dx = " << dx << ", dy = " << dy <<
147 → "\n\n";
148
149 out << "Проверка (невязка):\n";
150 out << "|f1| = " << abs(sys.f1(x, y)) << ", |f2| = " << abs(sys.f2(x, y)) << "\n";
151 if (abs(sys.f1(x, y)) < eps && abs(sys.f2(x, y)) < eps)
152     out << "Решение удовлетворяет системе с заданной точностью.\n";
153 else
154     out << "Невязка превышает заданную точность\n";
155 }
156
157 int main() {
158     vector<System> systems = {
159         {
160             "tg(xy + 0.1) = x^2; x^2 + 2y^2 = 1",
161             f1_sys1, f2_sys1,
162             df1dx_sys1, df1dy_sys1, df2dx_sys1, df2dy_sys1
163         },
164         {
165             "cos(x-1) + y = 0.5; x - cos(y) = 3",
166             f1_sys2, f2_sys2,
167             df1dx_sys2, df1dy_sys2, df2dx_sys2, df2dy_sys2
168         }
169     };
170
171     int choice;
172     cout << "Выберите систему:\n";
173     for (size_t i = 0; i < systems.size(); i++)
174         cout << i+1 << ". " << systems[i].description << "\n";
175     cout << "> ";
176     while (!(cin >> choice) || choice < 1 || choice > (int)systems.size()) {
177         cin.clear();
178         cin.ignore(numeric_limits<streamsize>::max(), '\n');
179         cout << "Некорректный ввод. Повторите:\n> ";
180     }
181 }

```



```

177     }
178
179     const System& sys = systems[choice-1];
180
181     double x0, y0, eps;
182     cout << "Введите начальное приближение x0 y0: ";
183     while (!(cin >> x0 >> y0)) {
184         cin.clear();
185         cin.ignore(numeric_limits<streamsize>::max(), '\n');
186         cout << "Ошибка. Введите два числа: ";
187     }
188     cout << "Введите погрешность: ";
189     while (!(cin >> eps)) {
190         cin.clear();
191         cin.ignore(numeric_limits<streamsize>::max(), '\n');
192         cout << "Ошибка. Введите число: ";
193     }
194
195     double xmin = x0 - 2.0, xmax = x0 + 2.0;
196     double ymin = y0 - 2.0, ymax = y0 + 2.0;
197     plotSystem(sys, xmin, xmax, ymin, ymax);
198
199     newtonSystem(sys, x0, y0, eps, cout);
200
201     return 0;
202 }

```

5. Выводы

В результате выполнения данной лабораторной работы я изучил численные методы решения нелинейных уравнений и их систем, а также реализовал на языке C++ методы хорд, секущих, простой итерации, половинного деления и Ньютона для одного уравнения, а также метод Ньютона для систем двух нелинейных уравнений. В ходе работы были выполнены все этапы: отделение корней графическим способом, выбор начальных приближений, уточнение корней с заданной точностью и проверка полученных решений.